

Lab 04 Mastering Simple Recursion

*A quick track to
learn coding
with confidence*

Recursive method - Coding is easier than tracing!
Q2-Q11 Q1

Q0. Review (Your task - Study the following and complete each task marked with ✍)

When we write a method, we should:

- First decide how the method will be used: give proper ①method name, ②parameter list, ③return type
A good start == 50% success / failure (Please circle the correct one)

- Then ④add the code in the method. If needed, ⑤make a method call to finish a detail step.

Please write 1-5 inside the circles:

```

○ void showDigits(int n)
{
○   if (n<10)
      System.out.print(n+" ");
   else
   {
○     showDigits(n/10);
      System.out.print(n%10+" ");
   }
}
  
```

Listing 1

- A parameter is / is not a special kind of local variable, which accepts a copy of the caller's argument.

Please choose

Q1. [Tracing]

Step 1. Download the given program and test it.

Testing 1:

Input n: 9
9

Testing 2:

Input n: 56
Please complete →

Testing 3:

Input n: 567
Please complete →

In line ____, there is a recursive method call.

Please complete

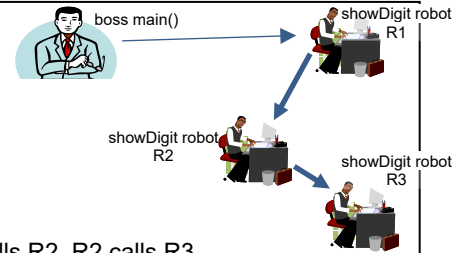
```

Line 5 void showDigits(int n) /* Based on Listing 1 */
Line 6 {
Line 7     if (n<10)
Line 8         System.out.print(n+" ");
Line 9     else
Line 10        {
Line 11            int leading = n/10;
Line 12            int right_most = n%10;
Line 13            showDigits(leading);
Line 14            System.out.print(right_most+" ");
Line 15        }
Line 16    }
Line 17
Line 18 void main(String[] args)
Line 19 {
Line 20     System.out.print("Input n: ");
Line 21     Scanner s = new Scanner(System.in);
Line 22     int n=s.nextInt();
Line 23     showDigits(n);
Line 24     s.close();
Line 25 }
  
```

Listing 2

Step 2. Read the following and fill in the 4 empty boxes

Consider line 23: `showDigits(n);`
line 13: `showDigits(leading);`



Each "method call" is similar to "calling a Robot"

- All calls to `showDigits` are to call "robots" which have the same design.
- `main()` is the big boss. At line 23: `main()` calls robot R1. At line 13: R1 calls R2, R2 calls R3.
- `main()` has his office space for `n`. Each robot R1, R2, R3 also has space for his own `n`, `leading`, and
- Note the process of **parameter passing**.
E.g., when R1 runs line 13 and calls R2:

R1's <code>leading</code>	====>	R2's <code>n</code>
argument		copy parameter

The complete story:

The big boss main calls the first robot `showDigit(567)`. This robot has his office: `n=567, leading=56, right_most=7`
[Then the `main()` waits.]

The first robot `showDigit(567)` will execute the second robot `showDigit(56)`. This robot has his office: `n=56, leading=5, right_most=6`
[Then the first robot waits.]

The second robot `showDigit(56)` will execute the third robot `showDigit(5)`. This robot has his office: `n=5`
[Then the second robot waits.]

The third robot `showDigit(5)` simply `System.out.print(n+ " ");`
The third robot finishes his job.

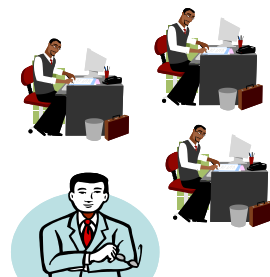
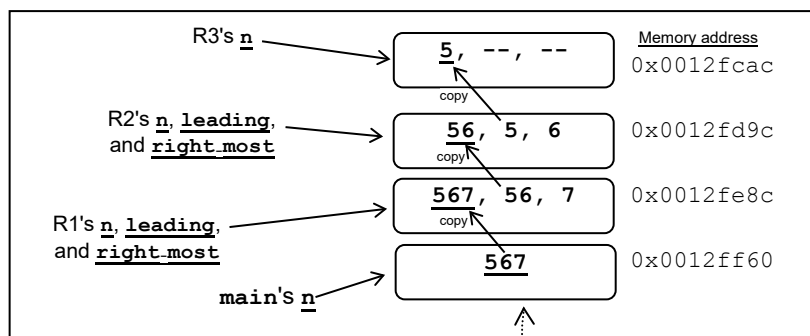
The second robot **resumes and continues**: `System.out.print(right_most+ " ");`
The second robot finishes his job.

The first robot **resumes and continues**: `System.out.print(right_most+ " ");`
The first robot finishes his job.

The big boss **resumes and continues**: whole program is finished. Final output: 5 6 7

Data Stack

- A data stack is a place in the computer memory, reserved for the execution of a program, **to hold the data of each method call**.



Explanation

- All local variables (including parameters) are kept in the **data stack** while the methods run.
- Eg. when R3 runs, 3 methods are waiting: `main()`, R1, R2.
The data of all these waiting methods the data stack at memory locations 0x0012ff60, 0x0012fe8c, 0x0012fd9c.
↑
"remain in" or "are removed from"?

Recursive method - Coding is easier than tracing!

Q2-Q11

Q1

Now: Practice how to write, not how to trace!!

↑ should pretend as a boss
who gives precise top-level logic

↑ When you design, don't spend time on tracing down the
method calls (this often scares you ☹ but not helps you.)

Solve Q2-11 using recursion. Assume that all integer data are +ve.

Q2. Get the largest digit from an integer n.

```
static int getLargestDigit(int n)
```

Sample run-down:

Input n: <u>56732</u> 7

Guideline:

Check and handle the **base case** √ i.e., simplest case which doesn't need recursion (here: $n < 10$)

Otherwise use recursion to solve a **subproblem** √ (here $n/10$), and then add **touch-up**.

Key trick: Be **confident** that the helper robot can solve the subproblem. √ Just trust! Don't hesitate.

Your task: Follow Lab04_guideQ2Q3Q4.pdf and finish Q2 in 5 minutes. Then review to digest the steps.

Q3. Determine whether integer x contains any even digit(s) : 0,2,4,6,8

```
static boolean containEven(int x)
```

Q4. Show the digits of integer x reversely (each followed by a space).

```
static void showDigitsReverse(int x)
```

Q5. Determine whether a given digit d exists in integer x.

```
static boolean containDigit(int x, int d)
```

Q6 Count the number of digits in integer x. (eg. 123456=>6, 8769=>4)

```
static int countDigits(int x)
```

Q7 Return the left-most digit in integer x (eg. 1234=>1, 8769=>8)

```
static int leftMostDigit(int x)
```

Have you finished the above with confidence? If no, ask for help and redo to digest well the proper approach.

- Q8. Determine whether integer x contains non-decreasing digits** (eg. 1234, 14789, 224466) i.e., whether the sequence of digits is "sorted" in ascending order.

```
static boolean containNonDecreasingDigits(int x)
```

↴ say 14789

↴ ie. 9

↴ ie. 8

- Hint: The two rightmost digits of x can be calculated as $\text{int } r1 = x \% 10$; and $\text{int } r2 = (x/10) \% 10$;

- Q9. Tail: Given an input integer x , return y which contains all digits in x except the left-most one.** e.g., 123=>23, 1234=>234. Assume $x > 10$ and x does not contain digit zero.

```
static int tail(int x);
```

- Q10. Determine whether the sequences of digits in 2 integers are opposite to one another** (eg. 123 and 321). You may make use of the previous methods `tail()` and `leftMostDigit()`. Assume that the input numbers do not contain digit zero.

```
static bool areOpposite(int x1, int x2);
```

- Q11. Return an integer which is the reversed version of the input integer x** (eg. 1234=>4321). You may make use of the previous recursive methods. Assume that the input numbers do not contain digit zero.

```
static int reverse(int x); [SEEMS DIFFICULT (But actually a short method)]
```

In Q1-Q11, the solutions are **1-way recursion** (ie. The method code makes direct call to only one "robot")

There are also 2-way recursion etc: e.g., the following example, which has been discussed as a lecture exercise.

```
public static int f(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
    {
        int f1= f(n-1);
        int f2= f(n-2);
        return f1+f2;
    }
}

public static void main(String[] args)
{
    System.out.println("Final result: "+f(5));
}
```